

HOOK DOCUMENT

DS 4002 | Case Study | University of Virginia

Do Star Ratings Tell the Whole Story?

Applying NLP to Movie Review Sentiment on Letterboxd

The Scenario

You've just been hired as a junior data scientist at a streaming startup. Your first assignment: the product team wants to build a smarter recommendation engine that goes beyond simple star ratings. They've handed you a dataset of tens of thousands of user reviews from Letterboxd, one of the most active movie communities on the internet, and they have a bold hypothesis: *the words people write reveal more about their true opinion than the number of stars they click.*

Your manager wants to know: can a state-of-the-art NLP model read a Letterboxd review and correctly predict whether the reviewer loved, hated, or felt lukewarm about the film? What is it about the way people write that confuses the model?

What You Will Do

- Load and explore a cleaned dataset of Letterboxd Top 250 film reviews.
- Apply a pretrained HuggingFace transformer model to classify review sentiment.
- Map numeric star ratings to sentiment categories and compare them to model predictions.
- Engineer text features (review length, emoji use, capitalization, language) and evaluate whether they explain where and why the model fails.
- Interpret your results and write up key findings.

The deliverable: A reproducible analysis, code, data, and a written report that evaluates how well NLP sentiment aligns with star ratings, and what textual features drive model accuracy. Details are in the rubric.

Why This Matters

Sentiment analysis is one of the most widely deployed NLP techniques in industry, used in product reviews, financial news, social media monitoring, and more. But pretrained models trained on Twitter or news data don't always generalize cleanly to casual, sarcastic, or culturally specific writing like movie reviews. Learning to diagnose *where* a model fails is just as important as knowing how to run it. This case study puts you at the intersection of NLP, model evaluation, and real-world messy data which are core skills for any data scientist.

Tools: Python | HuggingFace Transformers | pandas | scikit-learn | matplotlib